gv-scrollcast (#325)

An offline CLI that renders git-vista's **Print Graph** view — one very tall, light-background SVG sheet of the whole commit graph — to a full-height PNG, then pans a 1920×1080 camera down it into an MP4 the owner narrates over.

Deliberately outside the served app: it never binds a port, never touches an axum route, and never starts `./dev serve` or `trunk serve`. It drives a headless browser against a file on disk and shells out to `ffmpeg`.

What it does

```
capture   Print Graph sheet (HTML/SVG file) -> one full-height PNG,
          plus every commit node's pixel y-position
pacing    commit density -> speed curve -> a scroll timeline
          (dense stretches of history move slower, quiet ones faster)
chapters  chapters.txt sidecar (a YouTube-style timestamp list)
encode    timeline -> cropped frames -> H.264/yuv420p MP4, with +faststart
```

The pacing core (`pacing.rs`) is pure and host-tested — it decides how to scroll. Everything else is plumbing that carries that decision out to a real browser and a real encoder.

Prerequisites

No downloads, no network access at runtime. This crate resolves two external binaries in a fixed order — explicit override, then `$GV_SCROLLCAST_*` env var, then `PATH`, then a last-resort vendored copy — and fails at **startup** with a message naming every location checked if neither is found:

Binary	Env override	Resolution order
headless Chromium	`$GV_SCROLLCAST_CHROME`	override → env → `PATH` (`chrome-headless-shell`) → vendored Playwright copy
ffmpeg	`$GV_SCROLLCAST_FFMPEG`	env → `PATH` (`ffmpeg`) → vendored Playwright copy

On this box the vendored fallbacks are the Playwright cache under `~/.cache/ms-playwright/`. **That bundled `ffmpeg` is deliberately stripped** (built for Playwright's own webm/vp8 screen-recording, not general encoding) — it has no `libx264`, no `aac`, no PNG decoder, no MP4 muxer, and no `lavfi anullsrc` filter. `gv-scrollcast` checks for all five before doing any capture or encode work and refuses to start rather than fail partway through a multi-minute run. Point `$GV_SCROLLCAST_FFMPEG` at a full ffmpeg build to actually produce a video on this box.

The input file

`gv-scrollcast` cannot render the Print Graph sheet itself — there is no server route that serves it standalone, and starting one would mean binding a port (never done here; port

8080 is the owner's live session). So the sheet must already be on disk: open git-vista's Print Graph view, use "Print / Save PDF" → save as HTML instead (or save the page as HTML from the browser), and pass that file as `<input>`. A bare `.svg` extracted from that page also works, with caveats noted in `capture.rs`'s doc comment.

Usage

```
gv-scrollcast <input> [OPTIONS]

  <input>              the rendered Print Graph sheet (HTML preferred, SVG
accepted)
  --duration <secs>     target video length            [default: 240]
  --linear              constant-rate scroll, disables density pacing
  --date-overlay        burn a corner date marker into frames  [not yet implemented
— see below]
  --audio <path>        mux this audio file instead of a silent placeholder track
  --width <px>          rendered viewport width          [default: 1920 — see
below]
  --max-pivots <n>      cap pivot callouts               [default: 12 — currently
has no effect, see below]
  --out <dir>           output directory                 [default: ./out]
```

Worked examples

```
# The 4-minute default: density-paced scroll, silent placeholder audio track,
# written to ./out/scrollcast.mp4
gv-scrollcast sheet.html

# A constant-rate (non-adaptive) pass, 90 seconds, to a scratch directory
gv-scrollcast sheet.html --linear --duration 90 --out /tmp/gv-render

# With a narration track already recorded, muxed at its own natural length
# (never stretched/padded/truncated to fit — a length mismatch is reported,
# not silently corrected)
gv-scrollcast sheet.html --audio narration.wav --out ~/videos/git-vista-tour
```

A finished run reports, and writes:

```
gv-scrollcast: done
  video:      /home/tom/videos/git-vista-tour/scrollcast.mp4
  chapters:   /home/tom/videos/git-vista-tour/chapters.txt
  capture:    /home/tom/videos/git-vista-tour/capture.png
  7200 frames, 240.0s video
```

`chapters.txt` is a plain `timestamp label` list, one per line, in the exact format YouTube's video-description chapter parser accepts — paste it straight into the upload description.

Known gaps in this build

Three things worth knowing before assuming a flag does what its name suggests. All three are explained in full, with file:line citations, in `main.rs`'s top doc comment; this is the short version.

1. **Pivot callouts (merges, tags, month boundaries) don't fire yet.** Detecting them needs full commit metadata (refs, parents, message, time) aligned to each node's pixel position, and the capture stage currently extracts only the pixel positions. `chapters.txt` is still written every run, but today it only ever contains the mandatory `0:00 Start` line — there are no mid-video chapter marks or on-screen callout cards yet. `--max-pivots` is accepted and stored for when this is wired up, but has no effect in this build.
2. **`--date-overlay` is parsed but not drawn.** Burning a marker into every frame needs a hook in the frame-cropping step that doesn't exist yet. Passing the flag prints a warning and otherwise changes nothing about the output.
3. **`--width` must currently be `1920`.** The encoder's camera is a fixed 1920×1080; any other captured width is rejected at startup (before any capture work runs) rather than partway through the pipeline.

Determinism

Same machine, same pinned Chromium and ffmpeg binaries, same input file → the same PNG and the same video bytes. Not guaranteed across different Chromium/ffmpeg builds or different installed fonts — see `capture.rs`'s and `encode.rs`'s own doc comments for exactly what is and isn't covered and why.